Sistemas operativos

Jaime Nazar Anchorena

Contents

Unidad 1: Presentacion **2**
- Entorno de desarrollo . . . 2
- Crear procesos . . . 2
- `ps` . . . 3
- Copy on write . . . 3
- Sistemas operativos . . . 3
  - Administrador de recursos . . . 4
- Revision hardware . . . 5
  - Procesador . . . 5
  - Memoria . . . 5
  - Dispositivos I/O . . . 6
  - Booteo . . . 6
- Tipos de SO . . . 7
- System calls . . . 7

Unidad 1: Presentacion

Entorno de desarrollo

Trucos de bash:

- `ctrl + r` para buscar historial
- `!!` expande con el ultimo comando (`$sudo !!`)
- `file.{jpg, png}` expande a `file.jpg file.png`
- `$(...)` da el output de un comando (`touch file-$(date -I)`)
- `ctrl + a` para ir al comienzo de la linea
- `ctrl + e` para ir al final de la linea
- `cd -` vuelve al dir que estabas antes
- `fc` fix command, abre el ultimo comando en un editor y corre la version editada
- `type` indica de si algo es builtin, program, o alias (`type ping`)

En bin/ estan todos los comandos de administracion, para configurar el sistema. Por otro lado, en bin/ estan los comandos de uso general.

Crear procesos

Se pueden crear utilizando `fork`, syscall que la llamo una vez y vuelve dos veces, el proceso original y el que se creo.

Los dos procesos son identicos salvo el valor de retorno de `fork`. Si `fork` no puede crear un proceso(raro, pero puede pasar), devuelve -1.

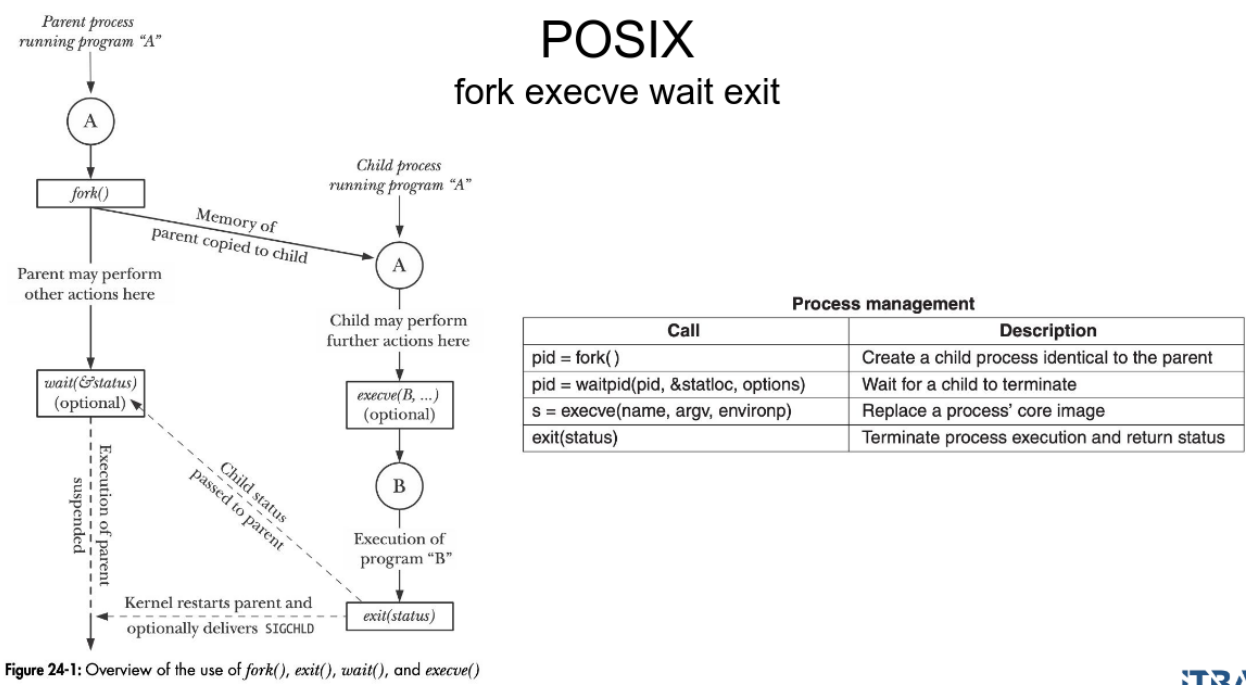

Figure 1: Pipeline fork

Como cada proceso puede crear varios procesos, pero tiene un solo padre, si quiero saber cual es el `pid` del

padre es necesario que el padre sepa cual es el `pid` del hijo. Esto se logra devolviendo el `pid` del hijo en el padre.

Es muy importante la llamada a `wait` porque sino queda un monton de procesos **undefined**, consume entradas en la tabla de procesos. Todas las variables que tenga un proceso original, incluyendo los punteros, el hijo los hereda. Es importante por ese motivo cerrar todos los **file descriptor**.

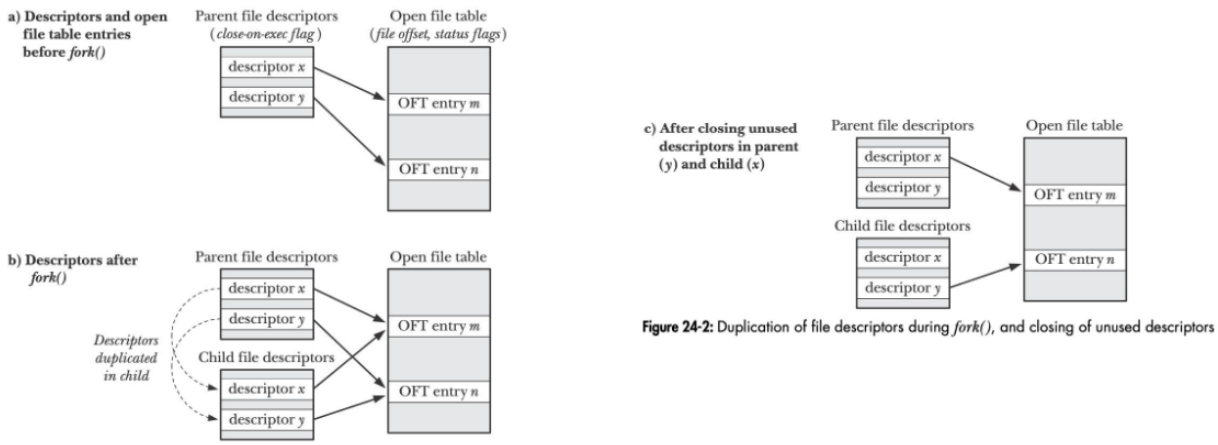

Figure 2: File descriptors fork

`ps`

Muy recomendado que lo aprendamos a usar bien para debuggear los TP. Se corre generalmente con `aux` pued indica: a+x -> mostrar todos los procesos, u -> incluir la columna con informacion del usuario de cada programa. Se puede combinar con otros arg(todo junto):

- Muestra los procesos que estan corriendo en el momento. Usando `w` como argumento(wide) muestra los argumentos que se utilizaron para correr cada programa.

- Si se corre con `e` va a mostrar las variables de entorno.
- Si se corre con `f` se mostrara un arbol de los procesos con caracteres ASCII

  Nota: Hay mas arg en la presentacion de campus.

Copy on write

Los procesos clones usan la misma memoria que el padre(heap, stack, memory maps), copiar esa memoria cada vez que usamos fork es lento y ocupa memoria. Entonces, linux permite que ambos procesos utilicen la misma memoria fisica y solo copia la memoria cuando alguno de los procesos intenta escribir. Linux implementa esta dandole la misma **page table** a cada programa, pero marcadas como **read only**. Entonces, cuando alguno intenta escribir ocurre una **page fault**.

Sistemas operativos

Primero planteemos que ocurriria si no hubiese un sistema operativo, entonces, `printf` no funciona, no tiene sentido. No hay drivers, no hay nada definido y no existe el SO que puede proveer el servicio de imprimir

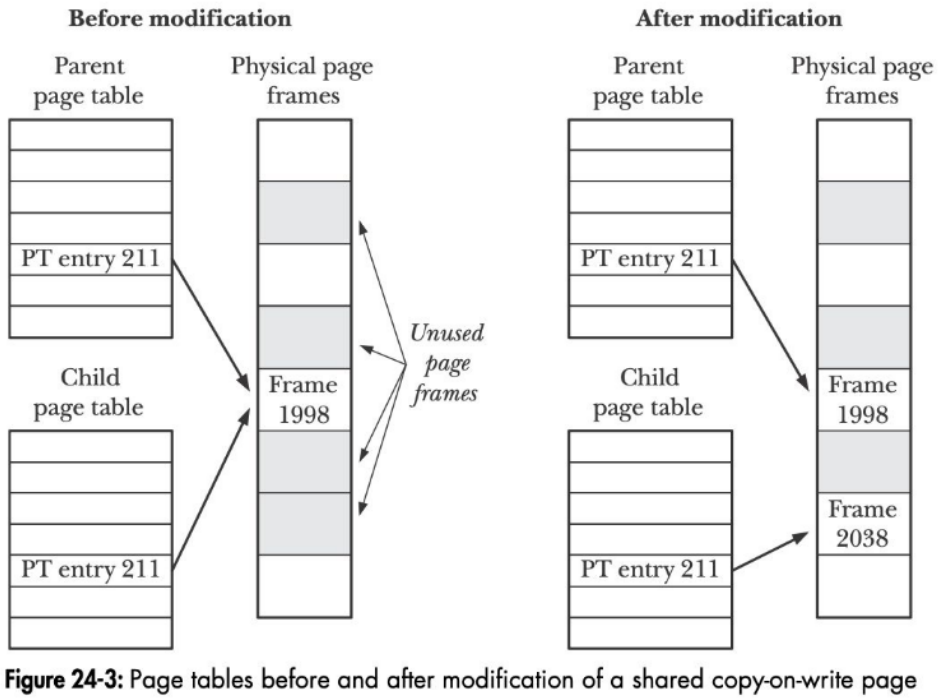

Figure 3: Representacion de copy on write

en pantalla.

En conclusion, el sistema operativo provee servicios(resuelve problemas) a nivel de llamar una funcion que haga algo, abstrayendo los detalles(no me importa en que hardware lo este corriendo).

Provee un conjunto obstracto y limpio de los recursos, administra estos recursos. Se requiere que el hardware tenga por lo menos dos modos de operacion(con distintos privilegios) para que funcione correctamente.

Oculta el hardware y ofrece una interfaz limpia, elegante y consistente al programador, las abtracciones son una de las claves para comprender SOs.

Administrador de recursos

Que pasa si 2 o mas procesos intentan imprimir en la impresora?

El SO es un ente centralizado que agrega cada impresion a un queue y garantiza que el acceso al recurso impresora sea correcto.

Administrar recursos incluye **multiplexar** estos recursos, mediante alguno de los siguientes metodos:

- Tiempo
- Espacio

Multiplexar trata de la distribucion de un recurso(temporal o espacialmente), por ejemplo, dividir el auditorio de SDT en clases mas chicas(espacial).

Revision hardware

Procesador

Un registro importante es el RFLAGS.

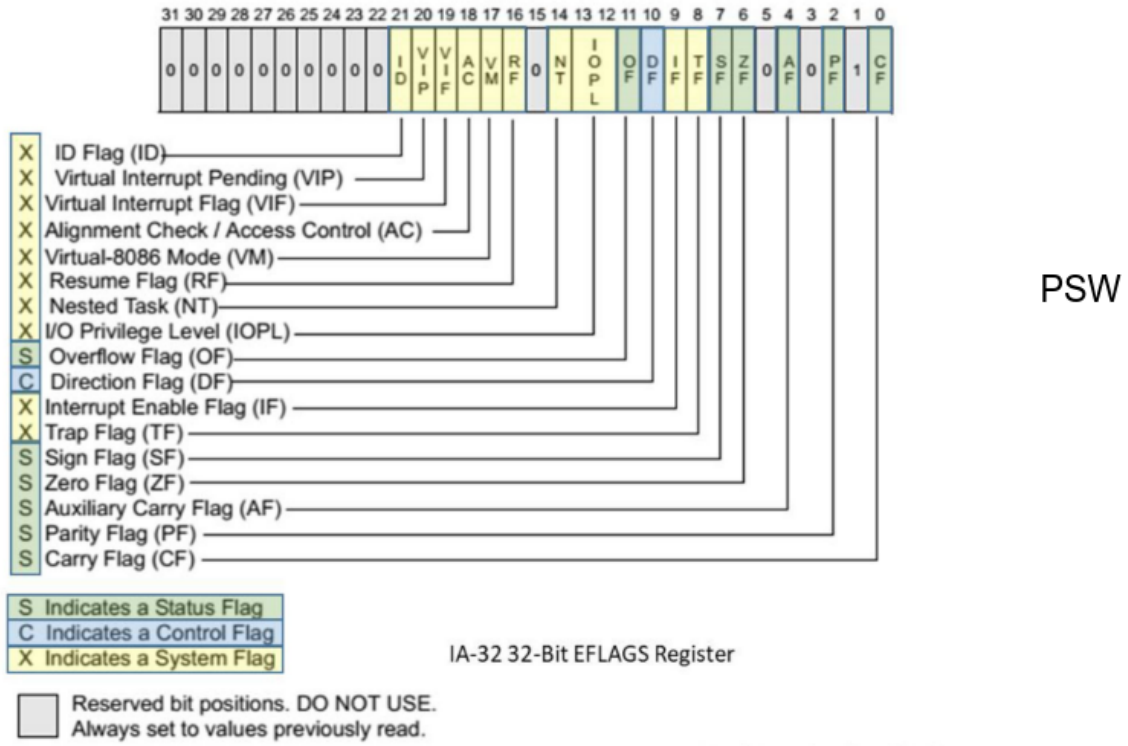

Figure 4: RFLAGS

Al iniciar el hardware, el stack estara vacio y el IP apuntara a la direccion de nuestro programa, pero primero hay que determinar cual es esa direccion.

Multithreading Mantener el estado de 2 threads e intercambiar entre ellos rápidamente cuando sea necesario. No es paralelismo.

Multicore Replicar los núcleos independientes. Pueden soportar múltiples threads.

> **Nota**: Ley de Moore es una aproximacion bastante acertada, pero llegamos a una limitacion de temperatura. Hoy en dia simplemente se replican los nucleos(muchos nucleos en un mismo procesador).

Memoria

Idealmente la memoria debería ser: - Extremadamente rápida (más que la ejecución de una instrucción) - Absurdamente grande (para albergar a todos los procesos juntos) - Asquerosamente barata

Obviamente esto es una utopia porque en la practica no se cumplen estas condiciones, por lo tanto, surgue una jerarquia de memoria(ver figura).

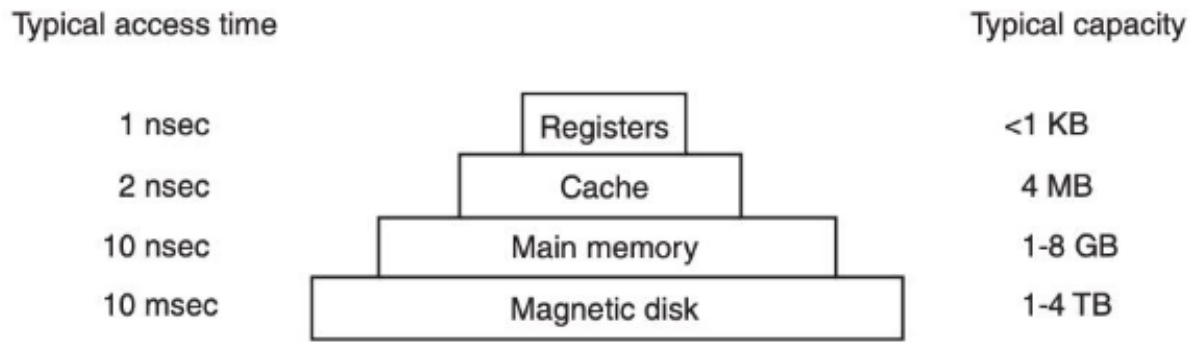

Figure 5: Jerarquia de los tipos de memoria

Dispositivos I/O

Un dispositivo consta de 2 partes:

- Controlador (abstracción)
- Dispositivo

Hay 3 formas de IO:

- Busy waiting(polling), en este caso, el cpu pregunta(es un derperdicio)
- Interrupción
- DMA (Direct Memory Access)

Proceso de interrupcion(?):

1. El driver le indica al controlador qué hacer. Este inicia la operación en el dispositivo.
2. El controlador señaliza al controlador de interrupciones (CI).
3. Cuando el CI está listo (otras interrupciones) setea el pin correspondiente en el CPU.
4. El CI escribe el número de dispositivo (vector de interrupciones) en el bus.

Booteo

- La placa madre posee un programa llamado BIOS (Basic I/O System)
- Flash RAM, no volátil pero actualizable por el SO
- La BIOS posee instrucciones para interactuar con la pantalla, teclado y disco

Proceso de booteo(una vez se determina cual es el dispositivo de booteo:

1. Chequear cuánta memoria tenemos y si ciertos dispositivos como el teclado están instalados y responden correctamente.
2. Escanea buses PCIe y PCI en busca de dispositivos instalados.
3. Determina el dispositivo de booteo.
4. Se carga el primer sector en memoria y se ejecuta
5. Este sector suele leer una tabla de particiones y se se carga un segundo sector
6. Se carga en memoria el SO y se ejecuta.

7. El SO consulta a la BIOS los dispositivos conectado

Tipos de SO

Ordenados de mas grandes a mas chicos.

- **Mainframe**: Mucho almacenamiento y memoria. Orientado a muchos procesos. Soporta trabajos por lotes, transacciones y timesharing. OS/390, tendencia a UNIX.
- **Server**: Computadoras personales o incluso mainframes. Impresoras, archivos o webs. Solaris, FreeBSD, Linux y Windows Server.
- **Multiproceso**: (Necesaria la separación)
- **Personal Computer**: Proveer soporte a un único usuario. Linux, FreeBSD, Windows y Apple's OS X.
- **HandHeld Computer**: Tablets y smartphones. Android y iOS
- **Embedded**: Electrodomésticos y teléfonos antiguos. ROM, no permite la instalación de aplicaciones. Embedded Linux, QNX y VxWorks.
- **Sensor-node**: Redes de nodos. Poseen CPU RAM y ROM. Sensan diferentes condiciones. Conexión inalámbrica. TinyOS
- **Real Time**: Necesitamos que una acción ocurra en un momento específico. hard vs soft. Industria, aviónica, milicia y multimedia. eCos y FreeRTOS
- **Smart Card**: Pagos electrónicos. Del tamaño de una tarjeta de crédito. Restricciones de potencia y memoria muy altas. Obtienen energía por inducción o al conectarse a un lector.

System calls

Como vimos en arqui, las syscall buscan:

- Proveer un conjunto abstracto y limpio de los recursos (hardware) -> entender qué hace el SO
- Administrar estos recursos -> transparente para el proceso
- Se puede ver como una simple llamada a función que además cambia a modo kernel
- El mecanismo para realizar una system call depende de la arquitectura y debe expresarse en assembler.

  Nota: Malloc puede derivar de una syscall, pero en realidad solo es necesario realizar una syscall cuando se acaba la memoria disponible para pedirle mas memoria al kernel.